TwinCAT Boot + HMI Server update scripts

Where	Run	What it does
Bench system	`Create-UpdatePackage.bat`	Uses ADS to switch TwinCAT to Config mode and stops the HMI Server (`TcHmiSrv`). Copies the Boot folder and HMI Server folder to `<USB>:\TwinCAT_Update` with a SHA-256 manifest, then restarts both.
Machine IPC	`Apply-Update.bat` (started by the PLC)	Finds and verifies the package on the USB stick. Uses ADS to switch to Config mode and stops the HMI Server. Backs up the old version, replaces the folders, verifies them, rolls back if anything fails, then reboots.

```
Apply-Update.bat / Create-UpdatePackage.bat   launchers
Config.ps1                                    paths, service name, options  <-- check this first
scripts\Common.ps1                            shared functions
scripts\New-UpdatePackage.ps1                 bench logic
scripts\Install-UpdatePackage.ps1             machine logic
```

One-time setup

1. **Check the paths in `Config.ps1`** against a real IPC. The first folder that exists is used, and the log shows which one.
   - Boot: `C:\ProgramData\Beckhoff\TwinCAT\3.1\Boot` (4026) or `C:\TwinCAT\3.1\Boot` (4024)
   - HMI: `C:\ProgramData\Beckhoff\TF2000 TwinCAT 3 HMI Server` or `C:\TwinCAT\Functions\TF2000-HMI-Server\service`
2. **No extra software is needed for ADS.** The scripts use the .NET ADS API (`TwinCAT.Ads.dll`) that comes with TwinCAT 3. They search the GAC and the TwinCAT install folders, preferring the self-contained v4.x DLL. On the machine, the DLL is found and connected before anything is stopped. If it can't be found, the update stops and nothing is changed. To check an IPC, especially a runtime-only (XAR) image:

   ```
   Get-ChildItem 'C:\TwinCAT','C:\Program Files (x86)\Beckhoff' -Recurse -Filter TwinCAT.Ads.dll
   -EA 0 | Select FullName
   ```

3. **On each machine**, copy this whole folder to a fixed local path, for example `C:\TwinCAT_UpdateTool\`. The PLC needs a path that doesn't depend on the USB drive letter. The USB stick only needs to hold the `TwinCAT_Update` package folder.

PLC trigger (Tc2_Utilities `NT_StartProcess`)

```
VAR
    fbStartUpdate : NT_StartProcess;
    bHmiUpdateButton : BOOL;   // linked to the HMI button
END_VAR

fbStartUpdate(
    NETID     := '',
    PATHSTR   := 'C:\Windows\System32\cmd.exe',
    DIRNAME   := 'C:\TwinCAT_UpdateTool',
    COMNDLINE := '/c C:\TwinCAT_UpdateTool\Apply-Update.bat',
    START     := bHmiUpdateButton,   // rising edge starts it
    TMOUT     := T#5S);
```

TwinCAT keeps running until the package has been fully validated. If the stick is missing or the package is bad, nothing is stopped and the machine keeps running. Once validation passes, the PLC and HMI go down and the IPC reboots about 10 s after the update finishes.

Optional arguments

Both launchers pass anything after them straight to their script, e.g. `Apply-Update.bat -NoReboot` or `Create-UpdatePackage.bat -Destination E:\`. The PLC trigger runs `Apply-Update.bat` with no arguments.

`Apply-Update.bat` (machine)

Argument	What it does
`-AllowVersionMismatch`	Applies the package even if the XAR / TF2000 versions differ from this machine. The mismatch is still logged. Manual use only.
`-NoReboot`	Doesn't reboot at the end. Restarts the HMI Server and puts TwinCAT back into Run mode instead.
`-PackagePath <folder>`	Uses this package folder instead of searching all drives for `\TwinCAT_Update`. Needed if more than one drive holds a package.
`-ConfigPath <file>`	Uses a different config file instead of `Config.ps1`.

`Create-UpdatePackage.bat` (bench)

Argument	What it does
`-Destination <drive or folder>`	Where to build the package, e.g. `E:\`. Default: the USB drive the script runs from, or the only USB drive present.
`-NoStop`	Copies without stopping TwinCAT / the HMI Server first. Not recommended: files can change mid-copy.
`-LeaveStopped`	Leaves TwinCAT in Config mode and the HMI Server stopped afterwards.
`-ConfigPath <file>`	Uses a different config file instead of `Config.ps1`.

Set the environment variable `NOPAUSE=1` to stop `Create-UpdatePackage.bat` waiting for a keypress at the end (useful when calling it from another script).

Result and logs (machine)

- `C:\TwinCAT_UpdateBackups\last_update_status.txt`: `SUCCESS` / `FAILED - nothing changed` / `FAILED - previous version restored` / `FAILED - ROLLBACK FAILED`
- `C:\TwinCAT_UpdateBackups\Logs\Apply_<time>.log`. A copy also goes to `<USB>:\TwinCAT_Update\Logs\`.
- `C:\TwinCAT_UpdateBackups\<time>_before_update\`: the old Boot and HMI folders. The last 5 are kept (`BackupsToKeep`).

Exit codes: 0 ok, 1 failed (nothing changed), 2 failed and rolled back, 3 rollback failed (the IPC is left in Config mode and **not** rebooted), 4 already running.

Manual rollback to an old version: copy `<time>_before_update\Boot` and `\HMI` back to their folders while TwinCAT is in Config mode and `TcHmiSrv` is stopped, then reboot. Or put that folder on a stick as `TwinCAT_Update`, set `"Purpose": "UpdatePackage"` in its `manifest.json`, and run Apply-Update. Each backup has its own `versions.json`, so the version check still applies.

Important behaviour

- **PLC persistent data is kept.** `PreservePersistentData = $true` means `*.bootdata` files are neither packaged nor overwritten, so each machine keeps its own retained values. If the new PLC software changes persistent variables, check how TwinCAT handles that before you rely on it.
- **Mirror copy.** The target folders end up exactly like the package. Stale files are removed, except excluded files, which are left alone.
- **Version check (exact match).** The bench writes `versions.json` into the package. It records the TwinCAT XAR version (registry `TcVersion`, e.g. `3.1.4026.27`) and the TF2000 HMI Server version (ProductVersion of the exe the `TcHmiSrv` service runs, e.g. `25.0.122`). Apply-Update reads the same values on the machine. If either differs, or can't be read, it stops with an error **before anything is stopped or changed**, and both versions are written to the log. The bench refuses to build a package if it can't read either version. Running `Apply-Update.bat -AllowVersionMismatch` by hand overrides the check; the PLC trigger never does.
- **Hardware binding.** The Boot folder holds the bench's I/O configuration. EtherCAT adapters are bound to a MAC address or PCI slot, so the bench and machines should be the same IPC model and use the same adapter binding. Licences are not part of the Boot folder and don't move with it.
- Incomplete packages are written as `TwinCAT_Update.partial` and are never picked up. The previous package on the stick is kept as `TwinCAT_Update_previous`.